Weeber × Shopify Integration — Coding Agent Reference Document

Project: Vocalist/Weeber — No-code Voice AI SaaS
Vertical: Shopify Merchants (Abandoned Cart Recovery + Order Confirmation)
Date: June 17, 2026
Author: Generated for coding agent handover — no prior history needed

---

1. WHAT WEEBER IS

Weeber (weeber.ai) is a vertical Voice AI SaaS for SMBs. It lets merchants connect their Shopify store and automatically trigger AI voice calls to:

1. **Abandoned cart recovery** — call customers 30 minutes after they abandon checkout
2. **Order confirmation** — call after order placed (optional, lower priority)

Tech stack: - **Backend:** Node.js + Express, Supabase (Postgres), hosted on Railway - **Frontend:** Vite + React + TypeScript (aurora-console) - **Voice:** ElevenLabs Conversational AI (primary provider) - **Telephony:** Twilio (outbound calls) - **Repo:** `/home/user/vocalist`

---

2. GOKWIK COMPETITIVE INTEL

GoKwik is the dominant Indian checkout intelligence platform. Understanding them is critical for Weeber's positioning.

What GoKwik Does

- Checkout optimization + COD (Cash on Delivery) risk scoring for Indian D2C brands
- 1-click checkout, address autofill, intelligent COD eligibility
- RTO (Return-to-Origin) prediction using ML on 600M+ buyer signals
- NDR (Non-Delivery Report) management with auto-retry workflows
- Embedded within Shopify via their own app + checkout extensions

GoKwik's Abandoned Cart Play

- GoKwik does SMS + WhatsApp cart recovery (no voice)
- Partners with brands like Boat, Mamaearth, Sugar Cosmetics

- Recovers 15-25% of abandoned carts via messaging flows
- **Gap:** Zero voice outreach — Weeber's wedge

Why This Matters for Weeber

- GoKwik owns checkout data for 50M+ Indian shoppers
- Weeber cannot access GoKwik data directly — must use Shopify native webhooks (`checkouts/create`, `checkouts/update`)
- Indian merchants already understand cart recovery ROI from GoKwik — Weeber has warm market
- Positioning: "GoKwik recovers carts with messages. Weeber does it with voice."
- Kyonara.com (Weeber's design partner) is a Shopify D2C jewelry brand — exact GoKwik customer profile

Indian Shopify Checkout Context

- Shopify's native abandoned checkout webhooks fire after 1 hour of inactivity
- GoKwik uses their own session tracking (faster, sub-minute)
- Weeber uses Shopify webhooks — 1-hour delay is acceptable for voice (not spam, feels human)
- TCPA compliance required for US merchants; India has no direct TCPA equivalent but TRAI regulations apply for commercial calls

---

3. VOCALIST CODEBASE — CURRENT STATE

3.1 Directory Structure (Relevant to Shopify)

```
/home/user/vocalist/
├── backend/
│   └── src/
│       ├── app.js                          # Express app entry
│       ├── modules/
│       │   ├── integrations/
│       │   │   ├── integration.routes.js   # Routes (MISSING OAuth wiring)
│       │   │   ├── integration.service.js  # Only has buildProvider()
│       │   │   ├── shopify.oauth.js        # Complete OAuth logic (NOT WIRED)
│       │   │   └── providers/
│       │   │       └── shopify.provider.js # Full Shopify API client
│       │   ├── campaigns/
│       │   │   ├── campaigns.routes.js     # Campaign + targets REST API
│       │   │   └── state-machine.js        # Call state machine
│       │   ├── calls/
│       │   │   └── call.service.js         # startOutboundCall() lives here
│       │   └── webhooks/
```

```
│       │       └── webhook.routes.js       # Vapi/Stripe/Twilio/ElevenLabs (NO Shopify)
└── frontend/                                # aurora-console (not in scope here)
```

3.2 What's Built

`shopify.provider.js` — Shopify API Client

- `ShopifyProvider` class instantiated with `{ shopDomain, accessToken }`
- Methods:
- `testConnection()` — GET /shop.json, returns shop object
- `getProducts(limit)` — GET /products.json
- `getCustomers(limit)` — GET /customers.json
- `getOrders(limit, status)` — GET /orders.json
- `lookupAbandonedCheckouts(limit)` — **DEPRECATED** REST endpoint (see issues)
- `setupWebhooks(baseUrl)` — registers `checkouts/create`, `checkouts/update`, `orders/create`, `orders/paid` webhooks
- `_handleCheckoutEvent(payload)` — **STUB** (logs only, no logic)
- `_handleOrderEvent(payload)` — **STUB** (logs only, no logic)
- `API_VERSION = "2024-01"` — **OUTDATED**, must be `"2025-01"`

`shopify.oauth.js` — OAuth Flow

- Complete OAuth implementation:
- `generateInstallUrl(shop, state)` — builds Shopify install redirect URL
- `handleCallback(shop, code, state, savedState)` — exchanges code for token
- `exchangeCodeForToken(shop, code)` — POST to Shopify token endpoint
- `verifyWebhookSignature(body, signature)` — HMAC-SHA256 verification
- Uses env vars: `SHOPIFY_CLIENT_ID`, `SHOPIFY_CLIENT_SECRET`, `SHOPIFY_SCOPES`
- Scopes needed: `read_orders,read_customers,read_checkouts,write_script_tags`
- **CRITICAL: This file is complete but imported/used nowhere**

`integration.routes.js` — Integration Routes

```
router.get('/shopify/connect', requireAuth, requireOrg, ...)   // starts OAuth
router.get('/shopify/callback', ...)                            // OAuth callback
router.get('/shopify/status', requireAuth, requireOrg, ...)    // check connection
router.delete('/shopify/disconnect', requireAuth, requireOrg, ...) // disconnect
```

- These route handlers exist but **do not import or call** `shopify.oauth.js` — they are stubs
- OAuth callback route is behind `requireAuth` middleware — **WRONG** (Shopify calls callback with no JWT)

`campaigns.routes.js` — Campaign System

- Full REST API: create, list, get, update, delete campaigns
- Campaign targets: `POST /campaigns/:id/targets` — bulk add phone numbers
- Campaign execution: triggers outbound call loop via state machine
- States: `QUEUED → DIALING → RINGING → IN_CALL → COMPLETED / FAILED / VOICEMAIL / RETRY_WAIT / DO_NOT_CALL`

`call.service.js` — Outbound Calls

- `startOutboundCall({ agentId, phoneNumber, orgId, metadata })` — places real Twilio call via ElevenLabs
- This is functional and used by campaigns
- **Missing:** dynamic variable injection (cart_items, cart_total, customer_name, abandoned_url)

`webhook.routes.js` — Webhooks

- Handles: Vapi status, Stripe billing, Twilio status, ElevenLabs events
- **Zero Shopify webhook handlers** — `POST /webhooks/shopify/checkout` does not exist

---

4. COMPLETE GAP LIST (P0 → P2)

P0 — Blocking (System Does Not Work)

#	Gap	File	Function/Route	Fix
P0-1	OAuth routes not wired	`integration.routes.js`	`shopify.oauth.js` never imported	Import `gener` `handl`
P0-2	OAuth callback behind requireAuth	`integration.routes.js`	`/shopify/callback`	Move `route` `requi`
P0-3	No `/api/integrations/shopify/connected` endpoint	`app.js` + `integration.routes.js`	Missing entirely	Add pu `X-Wee` token t table
P0-4	No Shopify webhook handlers	`webhook.routes.js`	Missing entirely	Add `P` `shopi` `webho`

#	Gap	File	Function/Route	Fix
				verify H provide
P0-5	`_handleCheckoutEvent` is a stub	`shopify.provider.js`	`_handleCheckoutEvent(payload)`	Upsert campa 30-min
P0-6	`_handleOrderEvent` is a stub	`shopify.provider.js`	`_handleOrderEvent(payload)`	Option confirm

P1 — High Priority (Feature Broken)

#	Gap	File	Function	Fix
P1-1	Deprecated REST endpoint	`shopify.provider.js`	`lookupAbandonedCheckouts()`	Remove or replace with webhook-captured data from DB
P1-2	Outdated API version	`shopify.provider.js`	`API_VERSION = "2024-01"`	Change to `"2025-01"`
P1-3	No dynamic variables in calls	`call.service.js`	`startOutboundCall()`	Pass `first_message` override + dynamic vars to ElevenLabs agent start
P1-4	No 30-min delay queue	DB schema / cron	Missing `scheduled_calls` table	Add table + cron worker to fire calls at `scheduled_at` time

P2 — Medium Priority (Quality / Scale)

#	Gap	File	Fix
P2-1	No webhook HMAC verification in route	`webhook.routes.js`	Call `verifyWebhookSignature()` from `shopify.oauth.js` before processing
P2-2		`shopify.provider.js`	

#	Gap	File	Fix
	No idempotency on checkout events		Check `checkout_id` already in DB before creating campaign target
P2-3	No DNC (Do Not Call) enforcement	`call.service.js`	Check DNC list before `startOutboundCall`
P2-4	No merchant call settings	DB schema	Store `call_delay_minutes`, `max_attempts`, `call_hours` per org
P2-5	No cart recovery script on storefront	Shopify theme	Optional: app embed to capture abandonment faster than Shopify's 1-hour webhook lag

---

5. END-TO-END FLOW (TARGET STATE)

```
MERCHANT ONBOARDING
──────────────────────────────────────────────────────────────────────
1. Merchant clicks "Connect Shopify" in Weeber dashboard
2. Frontend → GET /api/integrations/shopify/connect?org_id=xxx
3. Backend → shopify.oauth.js::generateInstallUrl(shop, state)
4. Redirect to Shopify OAuth consent screen
5. Shopify → GET /api/integrations/shopify/callback?code=xxx&shop=xxx&state=xxx
6. Backend → shopify.oauth.js::handleCallback() → exchangeCodeForToken()
7. Backend → call shopify.provider.js::setupWebhooks(baseUrl)
   - Registers: checkouts/create, checkouts/update, orders/create
8. Backend → upsert {shop, access_token, scopes, org_id} into integrations table
9. Dashboard shows "Connected ✓"

─── ALTERNATIVELY: weebersh Shopify app install ──────────────────
1. Merchant installs weebersh from Shopify App Store (unlisted)
2. weebersh handles OAuth natively (Remix app)
3. After install → weebersh POSTs to:
   POST https://api.weeber.ai/api/integrations/shopify/connected
   Headers: X-Weeber-Secret: <WEEBER_INTERNAL_SECRET>
   Body: { shop, access_token, scopes, org_id }
4. Weeber backend upserts token, registers webhooks

ABANDONED CART RECOVERY FLOW
──────────────────────────────────────────────────────────────────────
1. Customer adds to cart, starts checkout, leaves
2. Shopify fires: POST /webhooks/shopify/checkout
   Payload: { id, email, phone, total_price, line_items, abandoned_checkout_url }
3. webhook.routes.js receives it → verifies HMAC signature
4. Dispatches to shopify.provider.js::_handleCheckoutEvent(payload)
5. _handleCheckoutEvent():
```

```
   a. Extract: phone, email, customer_name, cart_items, cart_total, abandoned_url
   b. Upsert contact into contacts table (dedup on phone+org_id)
   c. Look up org's assigned Shopify voice agent (agent_id from integrations table)
   d. Insert into scheduled_calls:
      { org_id, agent_id, phone, scheduled_at: now()+30min,
        metadata: { cart_items, cart_total, customer_name, abandoned_url } }
6. Cron worker (runs every 5 min) queries scheduled_calls WHERE scheduled_at <= now()
7. For each due call → call.service.js::startOutboundCall({
     agentId, phoneNumber, orgId,
     dynamicVars: { customer_name, cart_total, cart_items, recovery_url }
   })
8. ElevenLabs agent speaks: "Hi {customer_name}, you left {cart_items} in your cart..."
9. Call result → update scheduled_calls.status (completed/failed/voicemail)
10. Dashboard shows call log with outcome

ORDER CONFIRMATION FLOW (P1)
─────────────────────────────────────────────────────────────────
1. Order placed → Shopify fires POST /webhooks/shopify/order
2. _handleOrderEvent() → immediate call (no delay)
3. Agent confirms order details, estimated delivery
```

---

6. WEEBERSH — SHOPIFY CONNECTOR APP

What it is: A minimal Shopify unlisted app. Its only job is OAuth handshake. No features, no UI beyond install confirmation.

Why separate: Shopify requires an app store listing for OAuth. Weeber's backend is not a Shopify app. weebersh bridges the gap.

Stack: Remix + `@shopify/shopify-app-remix` template
Repo: New repo — `github.com/I-invincib1e/weebersh`

One-Shot Prompt for Coding Agent (weebersh)

```
Build a Shopify unlisted connector app called "weebersh" using Remix + @shopify/shopify-app-r

PURPOSE: OAuth bridge only. After merchant installs the app, send their credentials to Weeber

AFTER INSTALL:
1. In the afterAuth hook (or session storage callback), POST to:
   https://api.weeber.ai/api/integrations/shopify/connected
   with headers: { "X-Weeber-Secret": process.env.WEEBER_INTERNAL_SECRET }
   with body: {
     shop: session.shop,
     access_token: session.accessToken,
     scopes: session.scope,
     org_id: new URL(request.url).searchParams.get("org_id") || null
   }
```

```
2. Show merchant a success screen: "Your Shopify store is connected to Weeber. Return to your

SCOPES REQUIRED: read_orders, read_customers, read_checkouts, write_script_tags

ENV VARS NEEDED:
- SHOPIFY_API_KEY
- SHOPIFY_API_SECRET
- SCOPES
- SHOPIFY_APP_URL
- WEEBER_INTERNAL_SECRET (shared secret with Weeber backend)

Use SQLite session storage for dev, Prisma+Postgres for production.
No other routes, no embedded app UI, no polaris components needed.
```

---

7. WEEBER BACKEND — ONE-SHOT PROMPTS FOR CODING AGENT

Prompt A: Wire OAuth Routes

```
File: /home/user/vocalist/backend/src/modules/integrations/integration.routes.js

CURRENT STATE: OAuth route stubs exist but shopify.oauth.js is never imported.
The callback route is incorrectly placed behind requireAuth middleware.

TASK:
1. Import shopify.oauth.js at top of integration.routes.js
2. Move /shopify/callback route BEFORE the router.use(requireAuth, requireOrg) line
3. In GET /shopify/connect handler: call shopify.oauth.generateInstallUrl(shop, state), redir
4. In GET /shopify/callback handler:
   - Call shopify.oauth.handleCallback(shop, code, state, req.session.oauthState)
   - On success: call shopify.provider.setupWebhooks(process.env.BASE_URL)
   - Upsert { shop, access_token, scopes, org_id } into integrations table
   - Redirect to frontend dashboard with ?connected=true
5. /shopify/status: query integrations table for org_id, return { connected: bool, shop }
6. DELETE /shopify/disconnect: delete from integrations table for org_id
```

Prompt B: Add Connected Endpoint (for weebersh)

```
File: /home/user/vocalist/backend/src/app.js

ADD before all authenticated routes:

app.post('/api/integrations/shopify/connected', express.json(), async (req, res) => {
  const secret = req.headers['x-weeber-secret'];
  if (secret !== process.env.WEEBER_INTERNAL_SECRET) {
    return res.status(401).json({ error: 'Unauthorized' });
```

```
  }
  const { shop, access_token, scopes, org_id } = req.body;
  if (!shop || !access_token) {
    return res.status(400).json({ error: 'Missing required fields' });
  }
  // Upsert into integrations table (Supabase)
  const { error } = await supabase
    .from('integrations')
    .upsert({ org_id, provider: 'shopify', shop_domain: shop, access_token, scopes, connected
              { onConflict: 'org_id,provider' });
  if (error) return res.status(500).json({ error: error.message });

  // Register Shopify webhooks
  const provider = new ShopifyProvider({ shopDomain: shop, accessToken: access_token });
  await provider.setupWebhooks(process.env.BASE_URL);

  res.json({ ok: true });
});
```

Prompt C: Add Shopify Webhook Handlers

```
File: /home/user/vocalist/backend/src/modules/webhooks/webhook.routes.js

ADD two new POST routes:

1. POST /webhooks/shopify/checkout
   - Read raw body (must use express.raw() middleware for HMAC)
   - Verify signature: shopifyOAuth.verifyWebhookSignature(rawBody, req.headers['x-shopify-hm
   - If invalid → 401
   - Parse JSON
   - Extract: shop domain from X-Shopify-Shop-Domain header
   - Look up org from integrations table by shop_domain
   - Call provider._handleCheckoutEvent(payload, orgId)
   - Return 200

2. POST /webhooks/shopify/order
   - Same HMAC verification
   - Call provider._handleOrderEvent(payload, orgId)
   - Return 200
```

Prompt D: Implement _handleCheckoutEvent

```
File: /home/user/vocalist/backend/src/modules/integrations/providers/shopify.provider.js

REPLACE the stub _handleCheckoutEvent with:

async _handleCheckoutEvent(payload, orgId) {
  // 1. Extract customer data
  const phone = payload.phone || payload.billing_address?.phone;
  const email = payload.email;
```

```
  const customerName = `${payload.billing_address?.first_name || ''} ${payload.billing_addres
  const cartTotal = payload.total_price;
  const cartItems = (payload.line_items || []).map(i => i.title).join(', ');
  const abandonedUrl = payload.abandoned_checkout_url;
  const checkoutId = payload.id;

  if (!phone) {
    console.log('[Shopify] No phone number in checkout, skipping', { checkoutId });
    return;
  }

  // 2. Idempotency check
  const { data: existing } = await supabase
    .from('scheduled_calls')
    .select('id')
    .eq('checkout_id', checkoutId)
    .single();
  if (existing) {
    console.log('[Shopify] Checkout already scheduled', { checkoutId });
    return;
  }

  // 3. Get org's Shopify voice agent
  const { data: integration } = await supabase
    .from('integrations')
    .select('agent_id, call_delay_minutes')
    .eq('org_id', orgId)
    .eq('provider', 'shopify')
    .single();

  const agentId = integration?.agent_id;
  const delayMinutes = integration?.call_delay_minutes || 30;

  if (!agentId) {
    console.log('[Shopify] No agent configured for org', { orgId });
    return;
  }

  // 4. Upsert contact
  await supabase.from('contacts').upsert(
    { org_id: orgId, phone, email, name: customerName },
    { onConflict: 'org_id,phone' }
  );

  // 5. Schedule call
  const scheduledAt = new Date(Date.now() + delayMinutes * 60 * 1000);
  await supabase.from('scheduled_calls').insert({
    org_id: orgId,
    agent_id: agentId,
    phone,
    checkout_id: checkoutId,
    scheduled_at: scheduledAt,
    status: 'pending',
```

```
    metadata: { customer_name: customerName, cart_total: cartTotal, cart_items: cartItems, re
  });

  console.log('[Shopify] Scheduled cart recovery call', { phone, scheduledAt });
}
```

Prompt E: Scheduled Call Cron Worker

```
Create file: /home/user/vocalist/backend/src/workers/call-scheduler.worker.js

PURPOSE: Poll scheduled_calls table every 5 minutes, fire due calls.

const { createClient } = require('@supabase/supabase-js');
const callService = require('../modules/calls/call.service');

const supabase = createClient(process.env.SUPABASE_URL, process.env.SUPABASE_SERVICE_KEY);

async function processDueCalls() {
  const now = new Date().toISOString();

  // Claim pending calls atomically
  const { data: dueCalls } = await supabase
    .from('scheduled_calls')
    .select('*')
    .eq('status', 'pending')
    .lte('scheduled_at', now)
    .limit(10);

  for (const call of dueCalls || []) {
    // Mark as processing first (idempotency)
    await supabase
      .from('scheduled_calls')
      .update({ status: 'processing' })
      .eq('id', call.id);

    try {
      await callService.startOutboundCall({
        agentId: call.agent_id,
        phoneNumber: call.phone,
        orgId: call.org_id,
        metadata: call.metadata,
        // Pass dynamic variables for ElevenLabs agent
        dynamicVars: {
          customer_name: call.metadata?.customer_name,
          cart_total: call.metadata?.cart_total,
          cart_items: call.metadata?.cart_items,
          recovery_url: call.metadata?.recovery_url,
        }
      });

      await supabase
        .from('scheduled_calls')
```

```
        .update({ status: 'dispatched', dispatched_at: new Date() })
        .eq('id', call.id);
    } catch (err) {
      console.error('[Scheduler] Call failed', { callId: call.id, error: err.message });
      await supabase
        .from('scheduled_calls')
        .update({ status: 'failed', error: err.message })
        .eq('id', call.id);
    }
  }
}

// Run every 5 minutes
setInterval(processDueCalls, 5 * 60 * 1000);
processDueCalls(); // run immediately on start
```

---

8. DATABASE SCHEMA ADDITIONS NEEDED

```
-- Add to Supabase via migration

-- scheduled_calls: delayed outbound call queue
CREATE TABLE scheduled_calls (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id        UUID NOT NULL REFERENCES organizations(id),
  agent_id      UUID NOT NULL REFERENCES agents(id),
  phone         TEXT NOT NULL,
  checkout_id   TEXT,                    -- Shopify checkout ID (idempotency key)
  scheduled_at  TIMESTAMPTZ NOT NULL,
  status        TEXT DEFAULT 'pending',  -- pending | processing | dispatched | failed | canc
  metadata      JSONB,                   -- { customer_name, cart_total, cart_items, recovery
  dispatched_at TIMESTAMPTZ,
  error         TEXT,
  created_at    TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX idx_scheduled_calls_due ON scheduled_calls (status, scheduled_at)
  WHERE status = 'pending';

-- Add to integrations table
ALTER TABLE integrations
  ADD COLUMN agent_id UUID REFERENCES agents(id),
  ADD COLUMN call_delay_minutes INTEGER DEFAULT 30,
  ADD COLUMN max_attempts INTEGER DEFAULT 2,
  ADD COLUMN call_hours_start INTEGER DEFAULT 9,   -- 9am merchant TZ
  ADD COLUMN call_hours_end   INTEGER DEFAULT 20;  -- 8pm merchant TZ

-- contacts table (if not exists)
CREATE TABLE IF NOT EXISTS contacts (
  id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
  org_id     UUID NOT NULL REFERENCES organizations(id),
  phone      TEXT NOT NULL,
  email      TEXT,
  name       TEXT,
  created_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE(org_id, phone)
);
```

---

9. ENV VARS NEEDED

Add to `/home/user/vocalist/backend/.env`:

```
# Existing
SHOPIFY_CLIENT_ID=...
SHOPIFY_CLIENT_SECRET=...
SHOPIFY_SCOPES=read_orders,read_customers,read_checkouts,write_script_tags
BASE_URL=https://api.weeber.ai

# New
WEEBER_INTERNAL_SECRET=<random 32-char string shared with weebersh>
```

---

10. IMPLEMENTATION PRIORITY ORDER

```
Day 1 (P0 — Unblock the system):
  1. Fix API_VERSION to "2025-01" in shopify.provider.js
  2. Implement Prompt A (wire OAuth routes)
  3. Implement Prompt B (add /api/integrations/shopify/connected endpoint)
  4. Implement Prompt C (add Shopify webhook handlers)
  5. Run DB migrations (scheduled_calls table + integrations columns)

Day 2 (P0 cont — Cart recovery works):
  6. Implement Prompt D (_handleCheckoutEvent full logic)
  7. Implement Prompt E (cron worker)
  8. Wire worker startup in app.js or separate process

Day 3 (P1 — Call quality):
  9. Update startOutboundCall to accept + forward dynamicVars to ElevenLabs
  10. Remove/replace deprecated lookupAbandonedCheckouts
  11. Test full flow: install weebersh → abandoned checkout → call fires

Day 4 (P1/P2 — Hardening):
  12. HMAC verification on all Shopify webhook routes
  13. Idempotency enforcement
  14. Merchant call settings UI (delay, hours, max attempts)
```

11. KEY DECISIONS (LOCKED)

Decision	Rationale
weebersh = OAuth bridge only, no features	Fastest path to Shopify install. Weeber dashboard handles everything else.
weebersh POSTs to `/api/integrations/shopify/connected` with shared secret	Avoids building full Shopify app inside Vocalist backend
30-min call delay default	GoKwik recovers in minutes via SMS. Voice at 30min feels human, not spammy.
Shopify webhooks for cart abandonment (not REST polling)	`lookupAbandonedCheckouts` REST is deprecated in 2024-04+ API
ElevenLabs dynamic variables for personalization	Pass `customer_name`, `cart_total`, `cart_items` at call start so agent speaks naturally
Cron worker polls every 5 min	Simple, reliable. Railway supports persistent workers. Redis queue is P3 over-engineering.

12. CONTACTS

- **Weeber GitHub:** github.com/Aurora-091/Vocalist
- **weebersh GitHub:** github.com/I-invincib1e/weebersh (to be created)
- **Design Partner:** Kyonara.com (Indian D2C jewelry, Shopify, in pilot)
- **Backend base URL:** https://api.weeber.ai
- **Brand:** Weeber (weeber.ai)

Document generated June 17, 2026. All file paths are absolute to the Vocalist monorepo at `/home/user/vocalist`. This document is the single source of truth for the Shopify integration coding sprint. No prior conversation history is needed — all context is embedded above.